Google Skills

Reading list

Congratulations!

You've completed course 4: Manage Agent Memory and State!

This course transformed your agents from stateless responders into intelligent assistants with state management. You've learned to give agents programmatic control over data, enabling exact tracking, dynamic behavior, and persistent preferences that conversation history alone cannot provide.

Your transformation:

Before course 4

Agents that respond to queries but rely entirely on conversation history—no programmatic access to data, no way to check exact values in code

After course 4

Agents with complete state management—programmatic control over exact values, automatic data injection via templating, persistent user preferences across sessions with appropriate persistence scopes

You now have the skills to build agents that remember, track, and adapt—the foundation for sophisticated stateful applications.

What you've accomplished

You started this lesson with basic LLM agents. You're finishing with stateful systems that can:

- **Access state programmatically** with `session.state` for exact value checking
- **Save data automatically** using `output_key` parameter
- **Inject state values** into instructions with `{var}` templating syntax
- **Persist user preferences** across sessions with `user:` namespace
- **Track exact metrics** like interaction counts without relying on LLM interpretation
- **Choose appropriate persistence scopes** (`temp:`, session, `user:`, `app:`)
- **Build dynamic agents** with instructions that adapt based on state values

Skills gained:

- Understanding state fundamentals and why it's essential for programmatic control
- Accessing state through `session.state` after running agents with Runner
- Using `output_key` for automatic response saving to state
- Using `{var}` templating for dynamic instruction injection
- Managing state namespaces for appropriate persistence scopes
- Deciding when to use each namespace based on data lifetime needs
- Implementing session initialization and user preference patterns

The journey:

Part 1

Understood session state and programmatic access with Runner

Part 2

Mastered `{var}` state templating for dynamic instructions

Part 3

Learned state namespaces and persistence scopes

The journey you've taken

Part 1

What is session state? (~15 minutes)

You learned what session state is and why it's essential for programmatic control—solving problems that conversation history alone cannot address.

Key concepts mastered:

- State provides programmatic access that conversation history doesn't
- `session.state` is a dictionary you access after running agents
- `output_key` automatically saves agent responses to state
- State enables exact tracking, templating, routing decisions, and cross-session persistence
- Use Runner and InMemorySessionService for programmatic state access

Key Insight:

"While LLM agents receive full conversation history by default, the key difference is that state provides **programmatic access**. Your code can check if `session.state.get('user_name'):` which is impossible with conversation history alone."

Code pattern

Python

```
from google.adk.agents import
LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import
InMemorySessionService

# Agent with output_key to save
response
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='name_extractor',
    instruction="Extract the person's
name from the message. Return ONLY
the name, nothing else.",
    output_key="user_name"  # Saves
response to state["user_name"]
)

# Setup and run
session_service =
InMemorySessionService()
session =
session_service.create_session(app_na
me="my_app", user_id="user1")
runner = Runner(agent=root_agent,
app_name="my_app",
session_service=session_service)

result = runner.run(user_id="user1",
session_id=session.id,
new_message="My name is Alex")

# Access state programmatically
print(session.state)  # {'user_name':
'Alex'}
if session.state.get("user_name"):
    print("✅ Name was successfully
extracted!")
```

Part 2

State templating (~15 minutes)

You learned to use `{var}` templating to inject state values into agent instructions, enabling dynamic agent behavior.

Key concepts mastered:

- `{var}` templating injects `state["var"]` into instructions before LLM call
- `{var?}` optional syntax doesn't error if key missing
- `{var?default}` provides default value if key missing
- ADK resolves templates automatically before sending to LLM
- Eliminates need for manual string formatting

Key Insight:

"ADK's `{var}` emplating automatically resolves state values before the instruction reaches the LLM. This enables dynamic, context-aware instructions without manual string manipulation."

Code pattern

Python

```
from google.adk.agents import
LlmAgent

# Agent with state templating
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='personalized_greeter',
    instruction="""
    You are a friendly assistant.

    User information:
    - Name: {user_name?there}
    - Preferred language:
{user_language?English}
    - Membership: {membership_tier?
free}

    {membership_tier?Your membership
level is: {membership_tier}}

    Greet the user warmly and offer
assistance.
    Respond in {user_language?
English}.
    """
)

# Manually set state (simulating
previous interaction)
session.state["user_name"] = "Alex"
session.state["user_language"] =
"Spanish"
session.state["membership_tier"] =
"premium"

# When agent runs, instruction
resolves to:
# "Hello Alex... Respond in
Spanish... Your membership level is:
premium"
```

Part 3

State namespaces and persistence (~15 minutes)

You learned about the four state namespaces that control how long data persists and who can access it.

Key concepts mastered:

- `temp:` state discarded after turn completes (invocation-scoped)
- Session state (no prefix) persists across turns, lost when session ends
- `user:` state persists across all sessions for that user
- `app:` state is global for all users
- Use narrowest scope needed (don't over-persist) State namespaces control lifetime and sharing

Key Insight:

“Use the **narrowest scope** that meets your needs. Don’t persist data longer than necessary. Temporary data should use `temp:`, conversation context should use session state, user preferences should use `user:`, and global config should use `app:`.”

Code pattern

Python

```
from google.adk.agents import LlmAgent

root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='namespace_demo',
    instruction="""
    You are a demo assistant showing state
namespaces.

    === App State (global for all users) ===
    App name: {app:name?Namespace Demo}
    App version: {app:version?1.0}

    === User State (persists across
sessions) ===
    User preference: {user:theme?not set}

    === Session State (persists this
conversation) ===
    Conversation topic: {topic?not set}

    === Temp State (current turn only) ===
    Current step: {temp:step?not set}

    Respond with a friendly message showing
these namespace values.
    """,
    output_key="response"
)

# Set all four namespace types
session.state["app:name"] = "Namespace Demo"
# Global for all users
session.state["app:version"] = "2.0"
# Global for all users
session.state["user:theme"] = "dark"
# Persists across sessions
session.state["topic"] = "state management"
# Persists this session
session.state["temp:step"] =
"initialization"     # Discarded after turn

# After agent runs:
# - temp:step is GONE (discarded after turn)
# - topic persists (session-scoped)
# - user:theme persists (user-scoped,
survives session end)
# - app:version persists (global, all users
see it)
```

What you can do now

With your mastery of state, you can build intelligent agents:

Access state programmatically

- Use Runner and InMemorySessionService for direct state access
- Check exact values with `session.state.get("key")`
- Make routing decisions based on state values
- Track metrics and counters programmatically

Manage state persistence appropriately

- Use `temp:` for intermediate processing data
- Use session state for conversation context
- Use `user:` for cross-session user preferences
- Use `app:` for global configuration
- Choose the right namespace for each use case

Use state templating

- Inject state values into instructions with `{var}`
- Handle missing keys with optional syntax `{var?}`
- Provide defaults with `{var?default}` pattern
- Create dynamic, context-aware instructions

Build stateful applications

- Save agent responses automatically with `output_key`
- Initialize sessions with default state values
- Track conversation progress across turns
- Remember user preferences across sessions
- Implement personalization based on user state

Real-world applications

Application: Personalized customer service agent

Combines state templating and namespaces for customer support:

Python

```
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

# Setup
session_service = InMemorySessionService()
session = session_service.create_session(app_name="support", user_id="customer1")

# Initialize state
session.state["user:name"] = "Alex Johnson"
session.state["user:tier"] = "premium"
session.state["user:language"] = "English"
session.state["app:business_hours"] = "24/7"
session.state["app:escalation_threshold"] = 3

# Agent with templating
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="""
    You are a {user:tier} customer support agent.

    Customer info:
    - Name: {user:name?Guest}
    - Support tier: {user:tier?standard}
    - Language: {user:language?English}

    Session info:
    - Message #{messages_in_conversation?1}
    - Issue: {issue_category?general support}

    Business hours: {app:business_hours}

    {needs_escalation?⚠️ ESCALATION THRESHOLD REACHED - Route to human agent if
unresolved.}

    Respond in {user:language} with personalized support.
    """,
    output_key="response"
)

runner = Runner(agent=root_agent, app_name="support",
session_service=session_service)
```

Use cases

1. Customer support chat with personalization
2. Technical support with tier-based access
3. Help desk automation with escalation tracking

Key takeaways

1. State provides programmatic control

While LLM agents receive full conversation history by default, state enables:

- Programmatic access: `if session.state.get("count") >= 3:`
- Exact values: Not LLM interpretation, but precise programmatic data
- Routing decisions: Make code-based decisions on exact values

Key principle: LLM can read both conversation history and state, but only state gives **your code** programmatic control.

2. output_key automatically saves agent responses

The pattern for automatic state saving:

- `output_key="result"`: Automatically saves agent response to `state["result"]`
- No manual code needed—ADK handles it automatically
- Works with all state namespaces (`output_key="user:pref"`)

Key principle: Use ADK's built-in `output_key` feature instead of manual state management.

3. {var} templating injects state into instructions

```python
# Static instruction (no state)
instruction="You are a helpful assistant."

# Dynamic instruction (with state)
instruction="""
You are a helpful assistant.
User language: {user:language?English}
Turn #{turn_count}
{premium_user?Access to premium features
enabled.}
"""
```

Templating syntax:

- `{var}` → Inject value (errors if missing)
- `{var?}` → Inject value (empty if missing)
- `{var?default}` → Inject value or default
- `{var?Conditional text}` → Show text only if var exists

Key principle: Use state templating to inject **exact programmatic values** into LLM instructions.

4. Four state namespaces control persistence scope

```python
state["temp:var"]      # Discarded after turn
state["var"]           # Discarded after session
state["user:var"]      # Persists across sessions for user
state["app:var"]       # Global for all users
```

Templating syntax:

- Need data only right now? → temp:
- Need data this conversation? → session (no prefix)
- Need data across all user's conversations? → user:
- Need data globally for all users? → app:

Key principle: Use the **narrowest scope** needed. Don't over-persist data.

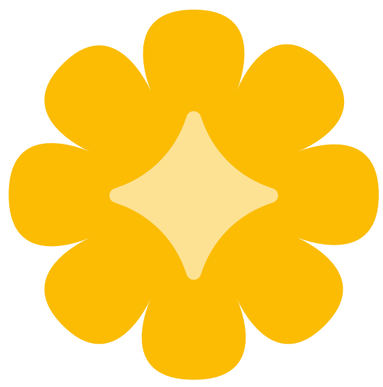
5. Access state with Runner pattern

```python
# Setup
session_service = InMemorySessionService()
session = session_service.create_session(app_name="app", user_id="user1")
runner = Runner(agent=agent, app_name="app", session_service=session_service)

# Run agent
result = runner.run(user_id="user1", session_id=session.id, new_message=msg)

# Access state programmatically
value = session.state.get("key", default)
if session.state.get("user_name"):
    # Make decisions based on state
    pass
```

Key principle: Use Runner to get programmatic access to session state after agent execution.



Best practices checklist

Apply these best practices to all your stateful agents:

State management

- [ ] **Use Runner pattern:** Access state programmatically with `session.state`
- [ ] **Use narrowest scope:** Don't persist data longer than necessary
- [ ] **Clean up temp state:** Use `temp:` for intermediate data
- [ ] **Document state keys:** Comment what each state variable represents

Templating

- [ ] **Use {var} templating:** Inject state into instructions dynamically
- [ ] **Handle missing keys:** Use `{var?}` for optional values
- [ ] **Provide defaults:** Use `{var?default}` when appropriate
- [ ] **Test with empty state:** Ensure defaults work correctly

Data saving

- [ ] **Prefer output_key:** Use instead of manual state management
- [ ] **Choose namespace carefully:** Match persistence needs
- [ ] **Validate state values:** Check types and ranges before use

Persistence

- [ ] **temp: for single turn:** Intermediate processing data
- [ ] **Session for conversation:** Turn counts, conversation context
- [ ] **user: for preferences:** Language, theme, settings
- [ ] **app: for global config:** Version, feature flags, business rules

Code patterns reference

Pattern 1: Basic state access

Python

```python
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

# Agent with output_key
agent = LlmAgent(
    instruction="Extract the main topic",
    output_key="topic"
)

# Setup
session_service = InMemorySessionService()
session = session_service.create_session(app_name="app", user_id="user1")
runner = Runner(agent=agent, app_name="app", session_service=session_service)

# Run
result = runner.run(user_id="user1", session_id=session.id, new_message=msg)

# Access state
topic = session.state.get("topic")
```

Pattern 2: State templating

Python

```python
# Agent with state templating
agent = LlmAgent(
    instruction="""
    Hello {user_name?there}!
    Preferred language: {user:language?English}

    {premium_user?✨ Premium features are enabled.}

    Respond in {user:language?English}.
    """
)

# Manually set state
session.state["user_name"] = "Alex"
session.state["user:language"] = "Spanish"
session.state["premium_user"] = True

# Instruction resolves to:
# "Hello Alex! ... Respond in Spanish. ✨ Premium features..."
```

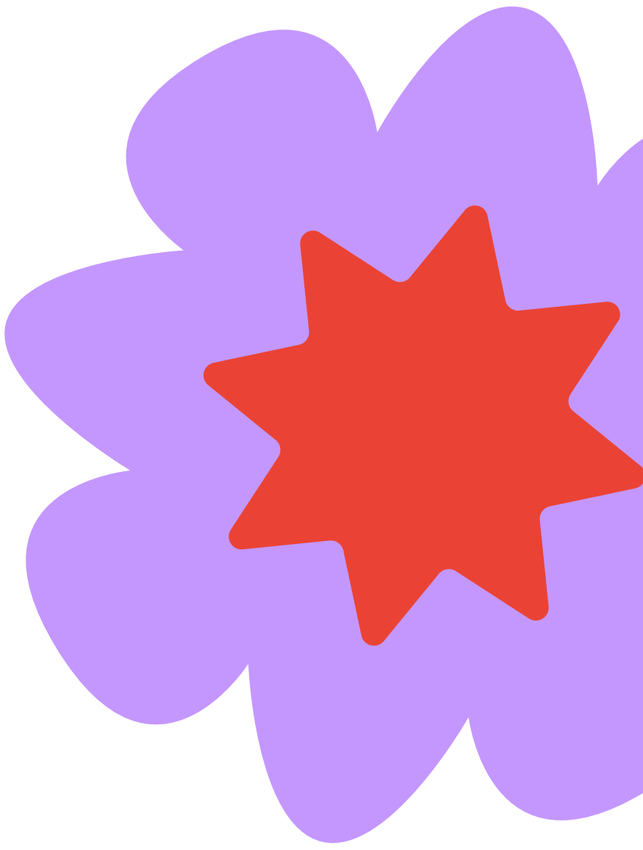
Pattern 3: All four namespaces

Python

```
agent = LlmAgent(
    instruction="""
    App: {app:name?MyApp} (v{app:version?1.0})
    User: {user:name?Guest} ({user:tier?free})
    Session: Turn #{turn_count?1}
    Processing: {temp:step?start}
    """
)

# Set state in different namespaces
session.state["app:name"] = "Support Bot"      # Global
session.state["app:version"] = "2.0"           # Global
session.state["user:name"] = "Alex"            # User-specific
session.state["user:tier"] = "premium"         # User-specific
session.state["turn_count"] = 5                # Session-specific
session.state["temp:step"] = "validating"      # Temporary

# After turn completes:
# - temp:step is GONE
# - turn_count persists (session)
# - user:name persists (user)
# - app:name persists (global)
```



Common pitfalls

Avoid these common mistakes:

Pitfall 1: Wrong persistence scope

```python
# ❌ WRONG: Using session state for user preferences
state["language"] = "Spanish"  # Lost when session ends

# ✅ RIGHT: Use user: for preferences
state["user:language"] = "Spanish"  # Persists across sessions
```

Pitfall 2: Not handling missing state keys

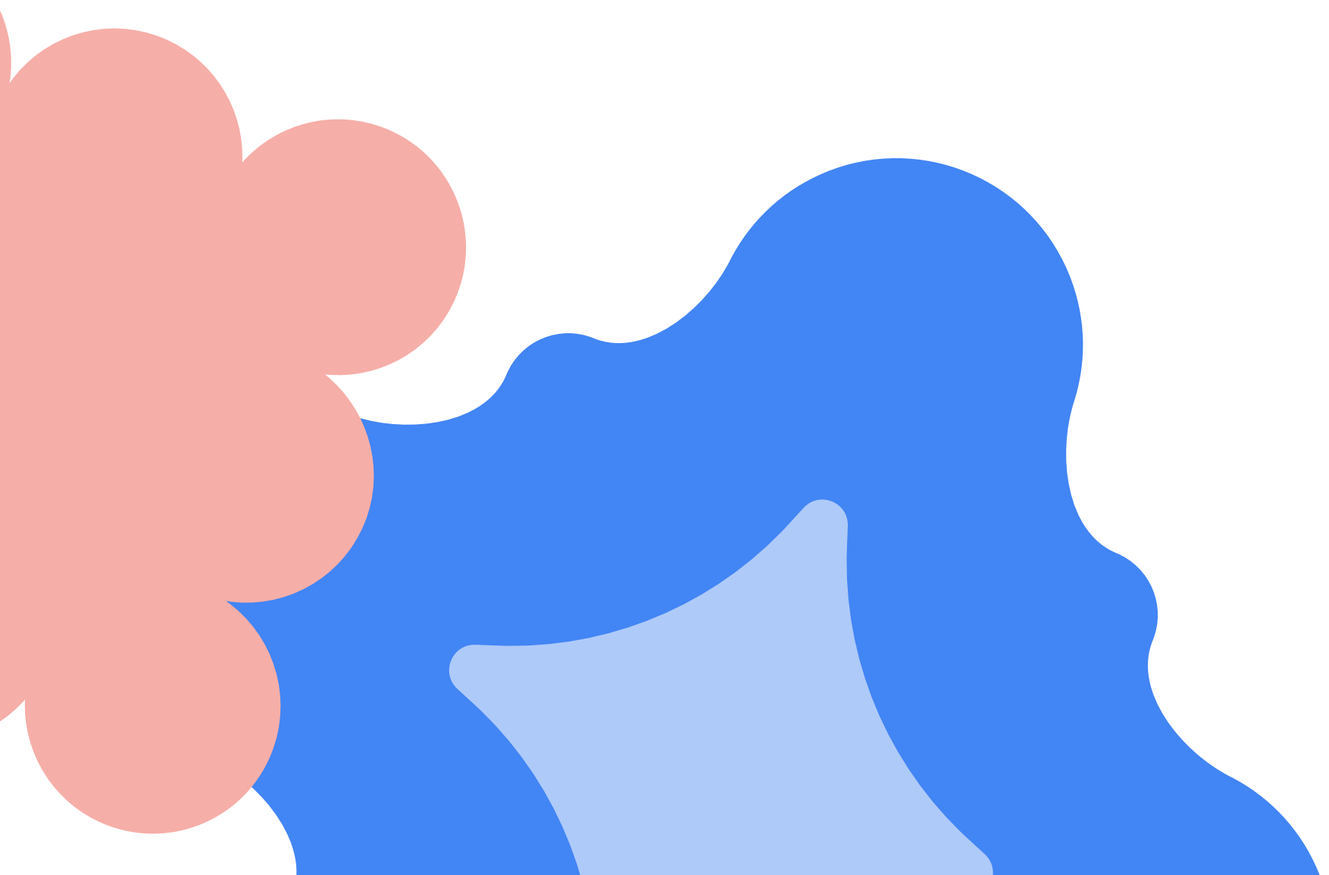
```python
# ❌ WRONG: Assuming key exists
instruction="Greet {user_name}"  # Errors if user_name doesn't exist

# ✅ RIGHT: Use optional syntax
instruction="Greet {user_name?there}"  # Defaults to "there" if missing
```

Pitfall 3: Over-persisting temporary data

```python
# ❌ WRONG: Persisting intermediate data
state["processing_step"] = "validating"  # Stays forever!

# ✅ RIGHT: Use temp: for temporary data
state["temp:processing_step"] = "validating"  # Gone after turn
```



Quick reference card

State namespaces

None

```
temp:var       → Current turn only (discarded after)
var            → Current session (discarded when session ends)
user:var       → User-specific (persists across sessions)
app:var        → Global for all users
```

State access

Python

```
# Read
value = session.state.get("key", default)

# Write
session.state["key"] = value
```

State templating

None

```
{var}               → Inject value (errors if missing)
{var?}              → Inject value (empty if missing)
{var?default}       → Inject value or default
{var?Conditional}   → Show text only if var exists
```

Data saving

Python

```
# Automatic saving
agent = LlmAgent(output_key="result")

# Automatic injection
agent = LlmAgent(instruction="Process {result}")
```

Decision tree

```
None
What type of data?

Structured values → STATE
  ├─ Current turn? → temp:
  ├─ Current session? → (no prefix)
  ├─ User-specific, all sessions? → user:
  └─ Global for all users? → app:
```

Community and support

Primary references

- **Introduction to Conversational Context**: Overview of session, state, and memory in ADK
- **Session Documentation**: Session lifecycle and SessionService implementations
- **State Management Guide**: State namespaces, persistence, and templating with {var} syntax
- **Memory Documentation**: Long-term memory and RAG-based retrieval across sessions
- **Vertex AI Session Management**: Managing sessions with ADK on Google Cloud

Questions? Post questions in the community forum.

Resources

From the community

- **Remember this: Agent state and memory with ADK** - Official Google Cloud Blog explaining sessions, state prefixes (user:, app:, temp:), and memory persistence patterns
- **How to use Sessions & State in Google ADK - Google Agent Development Kit for Beginners (Part 5)** - Complete 17-minute video tutorial covering session lifecycle, state management, SessionService implementations, and practical coding examples
- **Google ADK Masterclass Part 5: Session and Memory Management** - In-depth guide explaining how state enables agents to remember information without passing it in each message
- **State and Memory Management in Google ADK: A Practical Tutorial** - Hands-on tutorial showing how ADK uses prefixes to determine data persistence scope
- **Sessions and State Management in Google ADK - Building Context-Aware Agents (part 4)** - Practical guide to building agents that maintain context across conversations